

Laboratorio 3 - Tecnologías de Microprocesamiento

Ignacio Martínez
Ingeniería Mecatrónica
UTEC

Mercedes, Uruguay
ignacio.martinez.s@estudiantes.utec.edu.uy

Damian Bonilla
Ingeniería Mecatrónica
UTEC

Mercedes, Uruguay
damian.bonilla@estudiantes.utec.edu.uy

Matias Hernandez
Ingeniería Mecatrónica
UTEC

Mercedes, Uruguay
matias.hernandez@estudiantes.utec.edu.uy

Resumen—Este laboratorio abordó el diseño y la verificación de sistemas embebidos basados en el microcontrolador ATmega328P, enfatizando la adquisición de datos, control por PWM y comunicación serial. Se implementaron cinco aplicaciones: un plotter bidimensional gobernado por comandos UART; un control térmico con sensor LM35 que gestiona calefactor y ventilador; un control de motor DC con referencia analógica mediante potenciómetros; una matriz RGB WS2812B interactiva operada con joystick; y una cerradura RFID con almacenamiento de credenciales en EEPROM. La integración de lectura ADC, modulación PWM, protocolos UART/I²C/SPI y almacenamiento no volátil permitió validar el comportamiento de cada módulo y su coordinación dentro de un flujo de control confiable.

Index Terms—ATmega328P, UART, PWM, ADC, LM35, WS2812B, joystick, RFID RC522, EEPROM, I²C, SPI, control de motor, plóter.

I. INTRODUCCIÓN

Las tecnologías de microprocesamiento permiten desarrollar sistemas digitales capaces de sensor, procesar y actuar sobre variables físicas mediante el uso de microcontroladores y periféricos integrados. En este contexto, el presente laboratorio tuvo como propósito aplicar los conceptos de control, adquisición de datos y comunicación serial utilizando el microcontrolador ATmega328P como unidad central de procesamiento.

Durante las actividades se implementaron distintos módulos funcionales: un sistema de medición de temperatura con sensor LM35 y transmisión de datos hacia una interfaz en *Python*; un control de motores DC mediante modulación por ancho de pulso (PWM) con puente H L298N; un lector RFID RC522 para identificación de usuarios; un módulo de visualización con matriz RGB WS2812B controlada por un joystick; y un sistema de almacenamiento en memoria EEPROM mediante comunicación I²C.

Cada práctica permitió comprender de manera integrada el funcionamiento de los periféricos del ATmega328P, la importancia de la calibración y sincronización de señales, así como la interacción entre hardware y software en entornos embebidos. De esta forma, el laboratorio consolidó los fundamentos del control digital y la comunicación entre dispositivos en sistemas basados en microcontroladores.

II. OBJETIVOS

OBJETIVO GENERAL

Diseñar sistemas de control y adquisición de datos basados en el microcontrolador ATmega328P, integrando sensores, actuadores y comunicación serial para aplicaciones en tecnologías de microprocesamiento.

OBJETIVOS ESPECÍFICOS

- Implementar un sistema de medición de temperatura utilizando el sensor LM35 con transmisión de datos mediante UART hacia una interfaz en *Python*.
- Controlar la velocidad y dirección de motores DC mediante modulación por ancho de pulso (PWM) y puente H L298N.
- Leer tarjetas RFID RC522 para la identificación de usuarios y control de acceso.
- Visualizar colores en una matriz RGB WS2812B controlados mediante un joystick analógico.
- Almacenar y recuperar datos en memoria EEPROM utilizando el protocolo I²C.

III. MATERIALES UTILIZADOS

Para el desarrollo del laboratorio se emplearon componentes y equipos organizados por problema, de acuerdo con los requisitos de la consigna. A continuación, se detallan los materiales de hardware, así como el instrumental y software de apoyo para validación, simulación y registro de resultados.

Problema A: Control de Plotter mediante ATmega328P:

- Microcontrolador ATmega328P y placa/controladora provista en el laboratorio.
- Dos motores paso a paso (ejes X e Y).
- Drivers para motores paso a paso (interfaz de potencia con la controladora del plotter).
- Válvula solenoide para trazo (subida/bajada de lapicera/cuchilla).
- Seis sensores de límite (fin de carrera) para área de trabajo.
- Cableado y conectores según tabla de pines del sistema.
- Fuente de alimentación acorde a la controladora del plotter.

Problema B: Sistema de Control de Temperatura (Simulado y Físico):

- Microcontrolador ATmega328P.
- Sensor de temperatura LM35 con acondicionamiento de señal.
- Calefactor TIP31 (transistor).
- Motor DC usado como ventilador .
- Fuente de alimentación (5V para heater).

Problema C: Control de Motor con Referencia Analógica (Simulado y Físico):

- Microcontrolador ATmega328P.
- Dos potenciómetros 10kohms.
- Motor de DC 9V.
- Puente H L298N.
- Fuente de alimentación 9V.

Problema D: Matriz RGB con Joystick (Físico):

- Microcontrolador ATmega328P.
- Matriz de LEDs RGB 8x8.
- Joystick analógico.

Problema E: Cerradura RFID (Simulado y Físico):

- Microcontrolador ATmega328P.
- Lector RFID RC522.
- Pantalla LCD con interfaz I2C.
- LEDs indicador rojo y verde .
- Dos botones.

Instrumental y Software:

- Fuente de alimentación regulable y banco de pruebas.
- Multímetro y, cuando corresponde, osciloscopio para validación de señales.
- Protoboard, jumpers y conectores.
- Entorno de desarrollo en C para AVR (AVR-GCC/Microchip Studio) e interfaz UART (terminal serie).
- Herramienta de simulación para problemas B, C y E.
- Python o MATLAB para graficar: temperatura/acciones (Problema B) y curvas de referencia/posición/PWM (Problema C).
- Repositorio GitHub para versionado y trazabilidad del código.

IV. MARCO TEÓRICO

El estudio de las tecnologías de microprocesamiento permite comprender el funcionamiento e integración de los microcontroladores en sistemas de control, sensado y automatización. A través del ATmega328P, se exploran distintos periféricos como temporizadores, módulos ADC, comunicación serial y memoria EEPROM, aplicados al desarrollo de proyectos prácticos de control, visualización y respuesta ante el entorno.

El ATmega328P es un microcontrolador de 8 bits perteneciente a la familia AVR de Microchip Technology. Incorpora una arquitectura RISC mejorada con 32 registros de propósito general, lo que permite una ejecución eficiente de instrucciones en un solo ciclo de reloj. Opera a una frecuencia máxima de 20

MHz y dispone de 32 KB de memoria Flash para programas, 2 KB de SRAM y 1 KB de EEPROM. Entre sus principales periféricos se incluyen temporizadores de 8 y 16 bits, convertidor analógico-digital (ADC) de 10 bits, comunicación serial mediante USART, SPI e I²C, además de múltiples pines GPIO configurables como entrada o salida digital, en la Fig. 1 se encuentra un diagrama pinout del mismo. Gracias a estas características, el ATmega328P es ampliamente utilizado en aplicaciones de control, automatización y prototipado, siendo el núcleo de placas de desarrollo como Arduino Uno en la Fig.2 se encuentra una imagen del mismo.

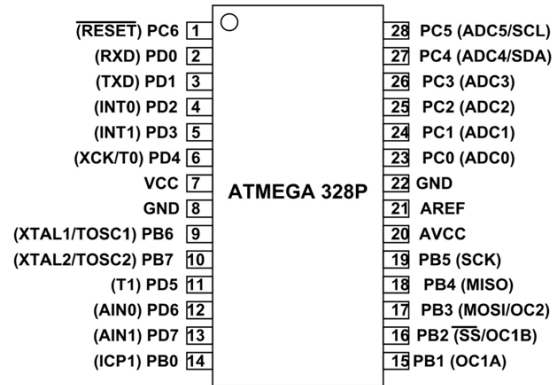


Figura 1: Microcontrolador ATmega328P

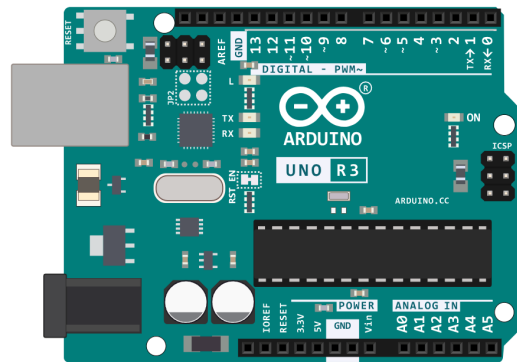


Figura 2: Plataforma Arduino UNO

La comunicación UART (Universal Asynchronous Receiver-Transmitter) es un protocolo serial asíncrono que permite el envío y recepción de datos entre dispositivos mediante una línea de transmisión (TX) y una de recepción (RX). Su funcionamiento se basa en la transmisión secuencial de bits, estructurados en tramas que incluyen bits de inicio, datos, paridad opcional y de parada, sin necesidad de un reloj compartido entre emisor y receptor.

Este tipo de comunicación es ampliamente utilizado en sistemas de microprocesamiento para la conexión entre microcontroladores y periféricos externos, como módulos Bluetooth, sensores o computadoras. Gracias a su simplicidad y fiabilidad,

UART resulta ideal para aplicaciones de monitoreo, depuración y control remoto. En la Fig. 3 se presenta un esquema representativo de este protocolo.

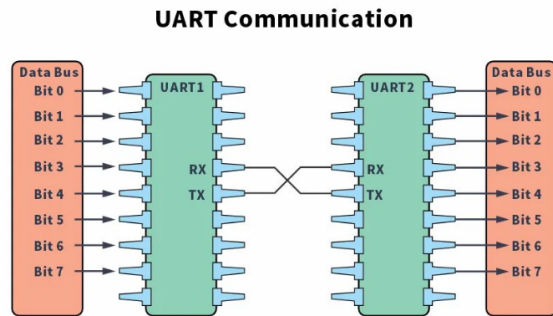


Figura 3: Esquema de comunicación UART

Un plotter es un dispositivo de impresión que, a diferencia de las impresoras convencionales, dibuja mediante el desplazamiento de un cabezal con pluma o un sistema de trazo lineal. Estos equipos permiten la representación precisa de gráficos, planos y figuras geométricas, por lo que son ampliamente utilizados en ingeniería, arquitectura y diseño.

En el ámbito académico y de laboratorio, los plotters pueden emplearse como sistemas de prueba para la programación y control de motores, integrando sensores y actuadores que permiten reproducir figuras en función de un programa predefinido. En la Fig. 4 se muestra un ejemplo de plotter utilizado en prácticas de mecatrónica y automatización.

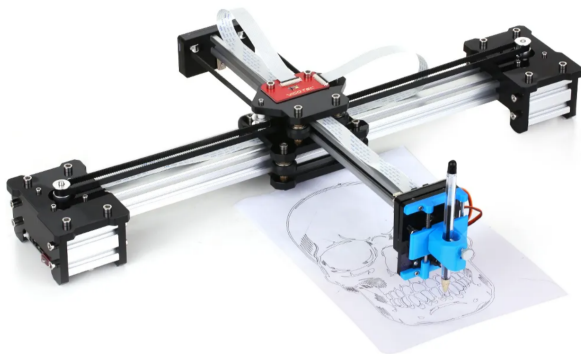


Figura 4: Plotter de trazo monocromático

El sensor de temperatura *LM35* es un dispositivo analógico de precisión que proporciona una salida lineal proporcional a la temperatura medida. Su principio de funcionamiento se basa en la variación del voltaje de salida en función de la temperatura ambiente, entregando una relación de $10 \text{ mV}/^{\circ}\text{C}$. Esto significa que, por ejemplo, a una temperatura de 25°C , la salida del sensor será de aproximadamente 250 mV .

El LM35 se alimenta típicamente con tensiones de entre 4 y 20 V y presenta una baja impedancia de salida, lo que facilita su conexión directa a las entradas analógicas del microcontrolador. Su ventaja principal frente a los termistores

o sensores de tipo resistivo es la linealidad de su respuesta, lo que simplifica la conversión analógica-digital y el cálculo de la temperatura en software. En la Fig. 5 se muestra la estructura básica y disposición de pines del sensor.



Figura 5: Estructura y disposición de pines del sensor LM35.

El transistor *TIP31* es un dispositivo semiconductor tipo NPN de propósito general utilizado para conmutación y amplificación de señales en aplicaciones de potencia moderada. Su funcionamiento se basa en el control de una corriente elevada entre el colector y el emisor mediante una corriente de base significativamente menor, actuando como un interruptor electrónico.

Este componente es capaz de manejar corrientes de hasta 3 A y tensiones de colector-emisor de hasta 40 V , lo que lo hace adecuado para el control de cargas como motores DC, ventiladores, calefactores o tiras de LEDs desde un microcontrolador.

El transistor suele conectarse en configuración de conmutador con una resistencia limitadora en la base, garantizando una corriente de activación segura y evitando daños en el microcontrolador. En la Fig. 6 se muestra el encapsulado TO-220 y la disposición de sus terminales.

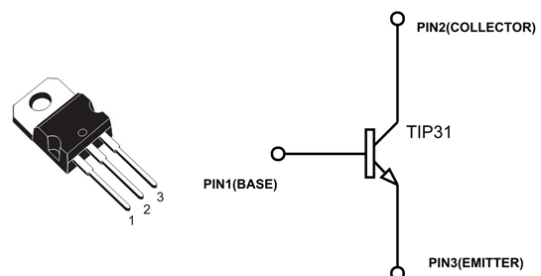


Figura 6: Encapsulado y disposición de pines del transistor TIP31.

Python es un lenguaje de programación de alto nivel ampliamente utilizado en el ámbito de la ingeniería para la adquisición, procesamiento y visualización de datos. Gracias a sus bibliotecas especializadas —como `PySerial`,

Matplotlib y CSV—, permite establecer comunicación serial con microcontroladores, registrar mediciones y generar gráficos en tiempo real.

En el contexto de este laboratorio, Python se empleó para desarrollar una interfaz de monitoreo que recibe los valores transmitidos por el microcontrolador a través de UART, permitiendo graficar parámetros como temperatura, velocidad o voltaje de salida. En la Fig. 7 se muestra el logo del lenguaje.



Figura 7: Logo de Python

La *Modulación por Ancho de Pulso* (PWM, por sus siglas en inglés) es una técnica empleada para controlar la cantidad de energía entregada a una carga mediante una señal digital. El principio de funcionamiento se basa en variar el ciclo útil (*duty cycle*) de una onda cuadrada, manteniendo constante su frecuencia. De este modo, se regula la potencia promedio aplicada a dispositivos como motores, LEDs o resistencias calefactoras.

En microcontroladores como el *ATmega328P*, las señales PWM se generan utilizando los temporizadores internos, que comparan el valor de un registro con un contador para determinar el tiempo en nivel alto de la señal. Ajustando este valor, se puede modificar el ciclo útil sin alterar la frecuencia. La Fig. 8 muestra el principio de funcionamiento de esta modulación.

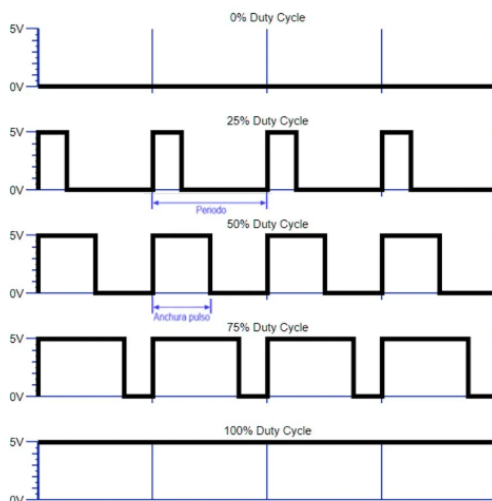


Figura 8: Representación gráfica del ciclo útil en una señal PWM.

El potenciómetro es un componente electrónico de tipo resistivo que permite variar manualmente la resistencia eléctrica entre sus terminales. Está formado por una resistencia fija y un cursor móvil (*wiper*) que se desplaza sobre ella, dividiendo el voltaje aplicado en una proporción determinada. De esta forma, se obtiene una salida analógica que varía de 0 V a V_{CC} , dependiendo de la posición del cursor.

En sistemas basados en microcontroladores, el potenciómetro se utiliza comúnmente como entrada analógica para ajustar parámetros como velocidad, brillo, volumen o temperatura. El valor de voltaje resultante es leído a través del conversor analógico-digital (ADC), permitiendo traducir la posición mecánica en un valor digital interpretable por el sistema. En la Fig. 9 muestra el pinout del potenciómetro.

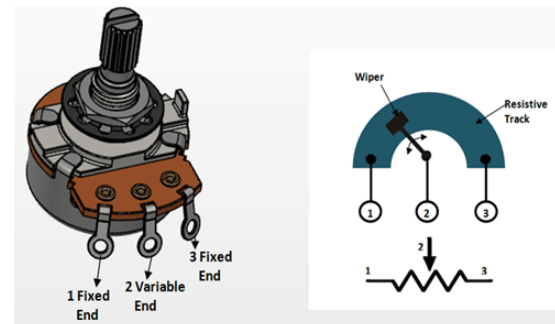


Figura 9: Pinout de un potenciómetro.

El *joystick analógico* es un dispositivo de entrada que permite medir desplazamientos en dos ejes perpendiculares (X e Y) mediante potenciómetros internos. Cada eje genera una señal analógica proporcional al movimiento, mientras que un pulsador integrado (botón SW) proporciona una entrada digital adicional al presionar el mando.

Estos dispositivos se utilizan ampliamente en control de robots, servomecanismos o interfaces gráficas, debido a su capacidad de traducir movimientos manuales en valores analógicos continuos. En los sistemas con microcontroladores como el *ATmega328P*, los ejes del joystick se conectan a entradas ADC, y el pulsador a una entrada digital con resistencia de *pull-up*.

En la Fig. 10 se presenta el pinout típico de un módulo de joystick analógico.

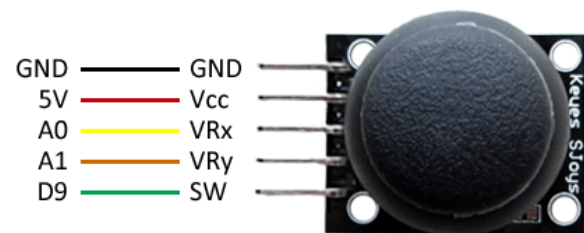


Figura 10: Pinout de un joystick analógico.

La matriz de LEDs RGB *WS2812B* es un conjunto de diodos emisores de luz direccionables individualmente, los cuales

integran en un mismo encapsulado los tres colores primarios (rojo, verde y azul) junto con un controlador interno. Este controlador permite gestionar de forma independiente el color y la intensidad de cada LED mediante una señal digital de un solo hilo (*one-wire*).

La comunicación se basa en un protocolo serial propietario que codifica la información de color mediante pulsos de ancho variable. Cada LED recibe la trama completa de datos, interpreta su porción correspondiente y retransmite el resto al siguiente, permitiendo el encadenamiento de múltiples LEDs en serie. En la Fig. 11 se muestra el diagrama de conexión de una matriz WS2812B típica.

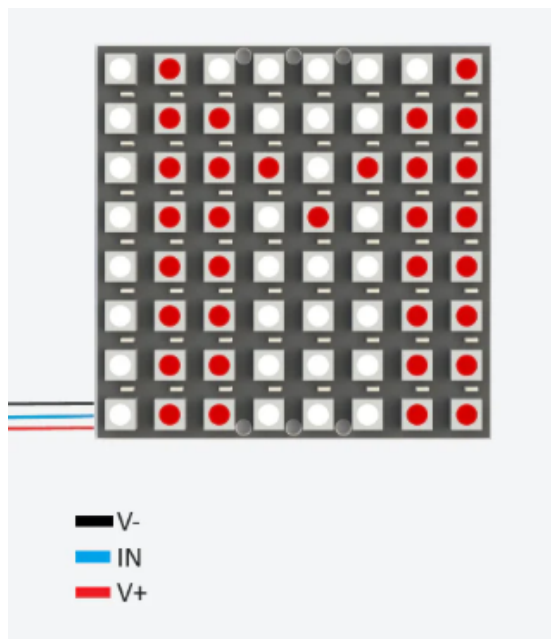


Figura 11: Diagrama de conexión de una matriz WS2812B.

El módulo *RFID RC522* es un lector/escritor de etiquetas de identificación por radiofrecuencia (RFID) que opera en la banda de 13.56 MHz, compatible con tarjetas y llaveros tipo *MIFARE*. Su función principal es la autenticación, lectura y escritura de datos almacenados en las tarjetas, permitiendo aplicaciones como control de acceso, registro de usuarios o identificación automática.

La comunicación entre el RC522 y el microcontrolador se realiza mediante el protocolo *SPI* (Serial Peripheral Interface), que permite una transferencia rápida de datos. El módulo integra una antena de cobre impresa y un circuito de demodulación para recibir las señales emitidas por la tarjeta. En la Fig. 12 se ilustra el módulo RC522 y su disposición típica de pines.

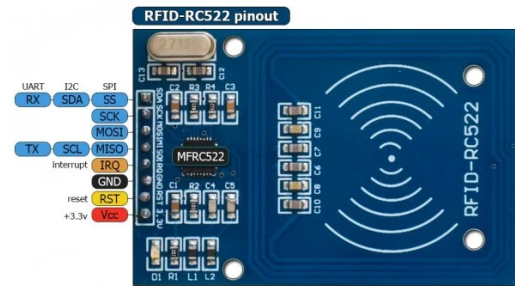


Figura 12: Módulo lector RFID RC522 y su pinout.

El LCD 16x2 es un display de cristal líquido con 16 caracteres por línea y 2 líneas de visualización. Cada carácter es representado por una matriz de 5x8 píxeles. Este tipo de pantalla es ampliamente utilizado en sistemas embebidos para mostrar información, como mensajes, resultados o estados de los sistemas, gracias a su capacidad de mostrar texto de manera clara y sencilla.

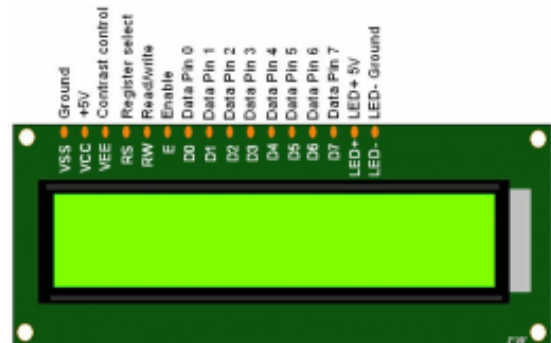


Figura 13: Pinout de un LCD 16x2.

El protocolo *Inter-Integrated Circuit* (I^2C) es un estándar de comunicación serial síncrona diseñado para la transferencia de datos entre múltiples dispositivos utilizando únicamente dos líneas: una de datos (*SDA*) y otra de reloj (*SCL*). Este protocolo permite conectar varios dispositivos maestros y esclavos en un mismo bus compartido, identificándolos mediante direcciones únicas.

En el contexto de los sistemas embebidos, el bus I^2C es ampliamente utilizado para conectar periféricos como memorias EEPROM, sensores de temperatura, pantallas LCD o convertidores A/D y D/A. Su estructura sencilla y la capacidad de controlar múltiples dispositivos con pocos pines lo hacen ideal para aplicaciones de microprocesamiento. En la Fig. 14

se muestra el esquema básico de comunicación entre un microcontrolador y un dispositivo esclavo mediante I²C.

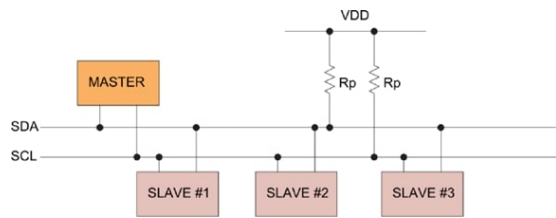


Figura 14: Esquema de comunicación entre maestro y esclavo mediante el protocolo I²C.

La memoria *EEPROM* (Electrically Erasable Programmable Read-Only Memory) es un tipo de memoria no volátil que permite almacenar información de forma permanente, incluso en ausencia de alimentación eléctrica. A diferencia de las memorias ROM convencionales, su contenido puede ser borrado y reescrito eléctricamente, lo que la convierte en un componente esencial para guardar configuraciones, parámetros de calibración o contraseñas en sistemas embebidos.

Este tipo de memoria se organiza internamente en celdas que pueden programarse individualmente, ofreciendo una alta durabilidad frente a múltiples ciclos de escritura y lectura. En microcontroladores y sistemas digitales, la comunicación con la EEPROM suele realizarse mediante protocolos seriales como I²C o SPI, lo que permite su fácil integración con otros dispositivos. En la Fig. 15 se muestra un ejemplo de chip EEPROM utilizado comúnmente en sistemas de control.

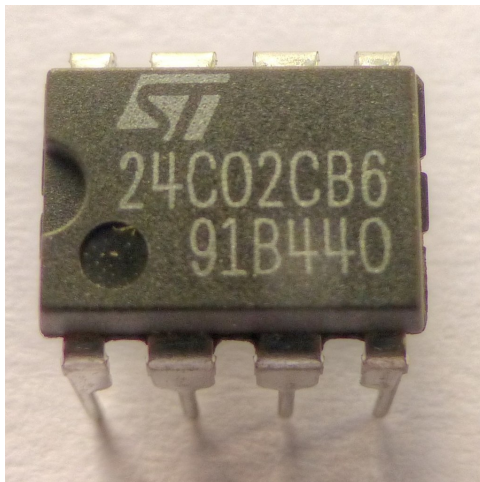


Figura 15: Ejemplo de chip de memoria EEPROM.

V. METODOLOGÍA

Para el desarrollo de este laboratorio, se comenzó con la revisión de las hojas de datos y manuales de los dispositivos utilizados, en particular el microcontrolador ATmega328P, el joystick analógico, la matriz de LEDs WS2812B, el motor DC, el sensor LM35 y la memoria EEPROM. Esta fase inicial permitió identificar las características clave de cada componente, tales como los rangos de operación, la distribución de pines,

los protocolos de comunicación (ADC, PWM, UART, I²C) y los principios de funcionamiento. A continuación, se procedió con las configuraciones necesarias tanto en hardware como en software, integrando las pruebas de simulación realizadas en Proteus y las validaciones experimentales en protoboard. De este modo, se establecieron las bases para implementar las metodologías de diseño, programación y pruebas que se describen en las siguientes secciones.

V-A. Problema A - Plotter

La implementación del plotter se basó principalmente en la programación del ATmega328P, conectado al tablero de control ARM Dolang, ya que la infraestructura mecánica y eléctrica del sistema ya estaba previamente montada y lista para su uso, como se puede observar en la Fig. 16. Además, se realizó el procedimiento de programación mediante el programador USBasp, lo cual permitió cargar el firmware necesario para controlar los movimientos del plotter y dibujar diferentes figuras. El objetivo fue desarrollar un programa que permitiera controlar los movimientos del plotter y dibujar diferentes figuras geométricas.

En primer lugar, se definió la tabla de pines del Arduino y la función de cada uno de ellos, indicando cuál controlaba el motor vertical (arriba/abajo), el motor horizontal (derecha/izquierda) y la activación del solenoide de la lapicera (subir/bajar). Esta tabla utilizada fue la siguiente:

Módulo AVR	ATmega328P	Etiqueta
PC0	PB3	CLK_X
PC2	PB4	DIR_X
PC3	PB5	EN_X
PC4	PC0	SOLENOID
PB10	PC3	CLK_Y
PB12	PC4	DIR_Y
PB14	PC5	EN_Y
PA8	PD2	LIMIT_YA
PA11	PD3	LIMIT_YD
PB6	PD5	LED

Tabla I: Tabla de conexiones del plotter.

El sistema permite dibujar figuras geométricas como triángulos, círculos, cruces y formas más complejas, activando los motores para realizar los movimientos.

Posteriormente, se planificó la ejecución de cada figura a dibujar:

- **Triángulo:** Se baja el solenoide (lapicera) al inicio, luego se desciende verticalmente, se mueve a la izquierda y finalmente se activa simultáneamente el motor vertical y horizontal para completar la diagonal, formando así el triángulo completo.
- **Cruz:** Para dibujar la cruz, primero se realiza una diagonal hacia arriba-derecha con la lapicera bajada. Luego se levanta el solenoide, se mueve a la izquierda, se baja nuevamente la lapicera y se dibuja la diagonal hacia abajo-derecha, completando la figura.
- **Círculo:** Debido a su curvatura, se dividió en múltiples pasos escalonados para aproximar la forma circular. Se

controlaron los motores horizontal y vertical alternando pequeños incrementos para simular la trayectoria curva.

- **Conejo:** Para dibujar el conejo, el sistema inicia posicionando el motor. Luego, el solenoide (lapicera) se baja y se mueve en varias direcciones para trazar las partes del cuerpo, las orejas y los detalles de la forma del conejo. Se realiza un movimiento secuencial, levantando y bajando el solenoide según sea necesario, para completar el contorno y los detalles de la figura.
- **Murciélago:** El dibujo del murciélago comienza con la colocación de la lapicera en la posición inicial, luego se mueve en varias direcciones para crear las alas y el contorno del cuerpo. Se alternan movimientos tanto verticales como horizontales, para formar las alas y la figura del murciélago. La figura se completa con una serie de pequeños movimientos para los detalles, siguiendo una secuencia controlada que simula el contorno del murciélago.

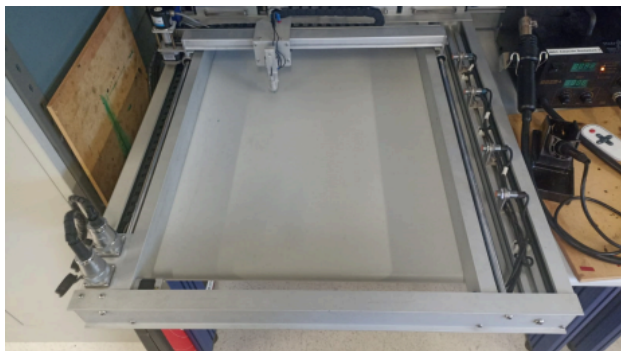


Figura 16: Plotter del Lab de Electrónica

V-B. Problema B – Sistema de Control de Temperatura

La implementación se abordó en tres fases encadenadas: simulación, montaje y visualización. En primer lugar, se construyó el modelo en *Proteus* empleando el LM35 como fuente de temperatura y representando el TIP31 de forma simbólica, ya que en el entorno virtual no es posible transferir calor real al sensor. La variable de temperatura del LM35 se barrió manualmente para emular escenarios de frío y calor, verificando la cadena de adquisición (ADC→), la modulación por ancho de pulso destinada al ventilador y la lógica ON/OFF del calefactor. Se comprobó además que al incrementar la temperatura simulada el duty del PWM aumentara progresivamente, y que al descender por debajo del umbral inferior el calefactor quedara activado, validando el comportamiento esperado del lazo.

Con el modelo validado, se utilizó el shield TCLAB montado sobre el Arduino, el cual ya incorporaba los componentes necesarios para la implementación del sistema de control de temperatura. Este shield incluye tanto el sensor LM35 para la medición de temperatura como los actuadores de calefactor TIP 31 y un pin para conectar el ventilador lo que simplificó la fase de montaje y permitió centrarse en la programación del control.

Se verificó que el sensor LM35 funcionara correctamente, leyendo sus valores a través del ADC del ATmega328P. También se comprobó el funcionamiento del ventilador y calefactor controlados por modulación por ancho de pulso (PWM), asegurando que respondieran de manera estable a las señales generadas por el temporizador del microcontrolador. Se cargó el firmware que ejecuta un ciclo periódico de lectura y promediado de la temperatura medida por el LM35, conversión a grados Celsius, y control del calefactor y ventilador.

Finalmente, se desarrolló una interfaz en *Python* para registro y visualización vía UART a 9600bps, mostrando temperatura, y el PWM del calefactor y ventilador en tiempo real.

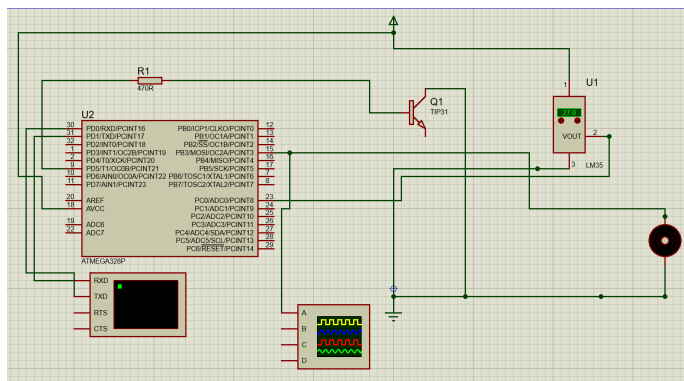


Figura 17: Esquema de simulación en *Proteus*

V-C. Problema C - Control de Motor

La implementación del sistema de control de motor con referencia analógica se estructuró en dos etapas principales: simulación y montaje físico. En ambas fases, se trabajó con el microcontrolador ATmega328P programado en C de bajo nivel, utilizando el ADC para la adquisición de la señal analógica proveniente del potenciómetro y PWM para controlar el motor.

En primer lugar, se realizó una simulación en *Proteus*, utilizando un motor simbólico para representar el comportamiento del sistema. El valor del potenciómetro, que simula la referencia de entrada, se modificó manualmente para verificar que el microcontrolador generaba el PWM adecuado en función de las variaciones de la señal analógica. Se conectó el potenciómetro a una entrada ADC del ATmega328P, mientras que la salida PWM controlaba el motor de forma simbólica en la simulación. Esta etapa permitió validar el flujo de lectura del ADC y la correcta generación de la señal PWM, ajustando su ciclo útil en función del valor de referencia. En la Fig. 18, se muestra el esquema de simulación en Proteus.

Una vez validada la simulación, se pasó al montaje físico en protoboard. Para el control del motor, se utilizó un puente H L298N, que permite controlar tanto la dirección como la velocidad del motor. El L298N se conectó al ATmega328P, recibiendo la señal PWM para controlar la velocidad del motor y las señales digitales para controlar la dirección. El potenciómetro, conectado a un pin ADC del microcontrolador, actuó como referencia de entrada.

Se verificó que el motor respondiera correctamente a los cambios de la señal PWM generada por el ATmega328P, logrando un control de velocidad suave y preciso en función de la variación del potenciómetro.

La visualización del comportamiento del sistema se realizó mediante la lectura de los valores del ADC y la aplicación de la señal PWM para controlar la velocidad del motor.

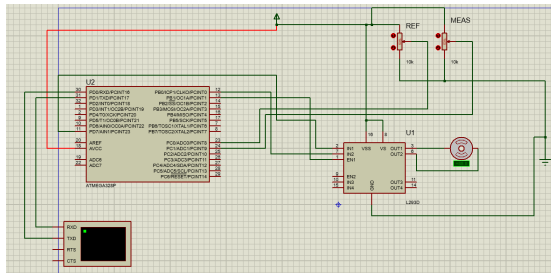


Figura 18: Esquema de simulación en *Proteus*: Control de motor por PWM con referencia de potenciómetro.

Problema D — Matriz RGB con Joystick

La implementación del sistema de control de la matriz RGB con joystick se estructuró en dos fases: montaje físico y programación del control de movimiento y color. En todo el proceso, se utilizó el microcontrolador ATmega328P programado en C de bajo nivel, utilizando el ADC para leer los valores de los ejes X e Y del joystick, y controlando la matriz de LEDs WS2812B para visualizar el movimiento y el color de un LED.

En primer lugar, se realizaron las conexiones físicas del joystick y la matriz de LEDs con el ATmega328P. El joystick analógico fue conectado a los pines del microcontrolador a través de las entradas del ADC, con el eje X conectado al canal A1 y el eje Y al canal A0. Además, el pulsador del joystick se conectó a un pin digital para detectar cuando el usuario presionara el botón para cambiar el color del LED. La matriz de LEDs WS2812B fue conectada a un pin digital de salida para controlar los LEDs de manera secuencial y enviar la información de color.

Una vez realizado el montaje físico, se programó el microcontrolador para que leyera los valores de los ejes del joystick mediante el ADC. Los movimientos de los ejes X e Y se tradujeron en el desplazamiento del LED dentro de la matriz de 8x8.

El color del LED se cambió aleatoriamente cada vez que el botón del joystick era presionado. Para ello, se generaron valores aleatorios para los componentes RGB del color utilizando una función basada en un generador de números aleatorios. El color del LED se actualizó en la matriz cada vez que se cambiaba, y la posición también se actualizó en cada movimiento del joystick.

Finalmente, se ajustaron los umbrales de movimiento para asegurarse de que el LED no se moviera de manera errática cuando el joystick estaba en su posición central. También se verificó que el cambio de color funcionara correctamente al

presionar el botón del joystick y que la matriz se actualizara en tiempo real.

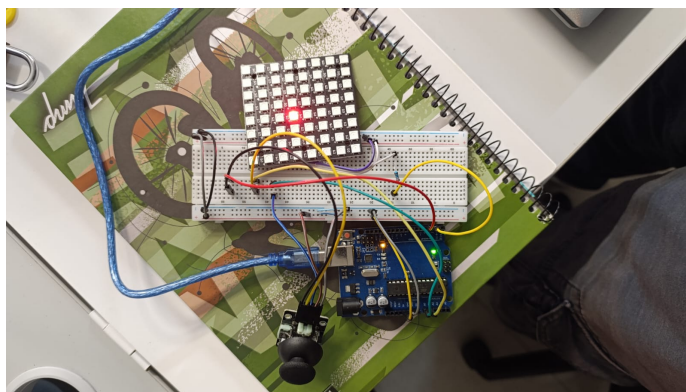


Figura 19: Montaje del control de matriz de LEDs con joystick.

V-D. Problema E - Cerradura RFID

La metodología seguida para la implementación de la cerradura RFID con el microcontrolador ATmega328P se estructuró en varias fases: diseño y simulación, montaje físico, programación de las funcionalidades principales y pruebas de validación. En todas las fases se trabajó en C de bajo nivel, utilizando SPI para el lector RC522, I²C para la LCD 16x2 (PCF8574), y EEPROM para almacenar UIDs autorizados; además, se usó UART para depuración y trazas.

Inicialmente, se diseñó el circuito en *Proteus*, integrando el RC522 como periférico SPI del ATmega328P y la LCD por I²C para la indicación de estado. Se verificó la secuencia de inicialización del lector y la lectura del UID de tarjetas/llaveros. Los LEDs (verde/rojo) y los botones (ACTUALIZAR/BORRAR) se incorporaron para indicar resultado de autenticación y para gestionar el alta/baja de tarjetas en memoria. En simulación se comprobó que, ante una tarjeta no registrada, el sistema mostrara “Acceso denegado”, y que, tras el alta, la misma tarjeta pasara a “Acceso concedido”.

Una vez validado el diseño en simulación, se procedió al montaje físico en protoboard. El RC522 se alimentó a 3.3V y la LCD a 5V, manteniendo masa común y cableado corto para SPI. La comunicación I²C se configuró a 100kHz y se habilitaron *pull-ups* en las líneas SDA/SCL si fue necesario. Los botones se conectaron a entradas con *pull-up* interno para activar a nivel bajo. Se verificó la estabilidad de lectura del UID y la coherencia de los mensajes en la LCD durante el uso normal.

En paralelo, se programaron las funcionalidades principales del sistema en C, de acuerdo con los siguientes requerimientos:

- Autenticación por UID: El microcontrolador consulta el RC522, obtiene el UID y lo compara contra una lista blanca almacenada en EEPROM. Si coincide, habilita el acceso (LED verde y mensaje en LCD); si no, lo deniega (LED rojo).
- Alta y baja de tarjetas: Mediante el botón ACTUALIZAR se registra el UID presente en el lector (evitando

duplicados); con BORRAR se elimina el UID leído o se limpia la lista según el modo configurado. La LCD confirma cada operación.

- Mensajería y trazas: La LCD 16x2 muestra estados (*Listo, Concedido, Denegado, UID guardado/eliminado*); por UART se envían trazas de diagnóstico (UID leído, resultado y errores).
- Temporizaciones y robustez: Se aplicó debounce por software a los botones, timeouts de lectura del RC522 y un backoff breve tras accesos denegados para evitar lecturas repetidas. Opcionalmente se activó una salida de “cerradura” por tiempo fijo al conceder acceso.

Finalmente, el sistema fue probado de extremo a extremo, validando la detección estable de UIDs, el alta/baja en EEPROM y la coherencia entre LEDs, LCD y trazas UART. Se ajustaron tiempos de *debounce* y de *timeout* hasta lograr una operación fluida sin falsas detecciones.

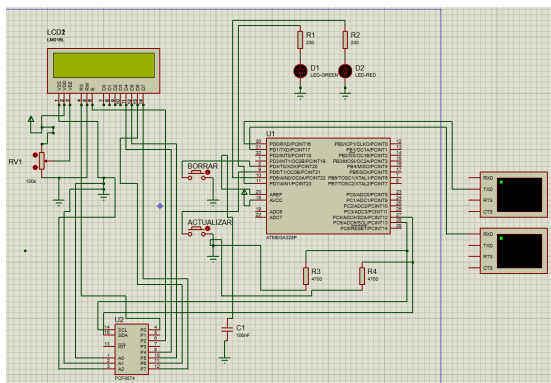


Figura 20: Esquema de simulación de la cerradura RFID en Proteus.

VI. RESULTADOS

VI-A. Plotter

La implementación del plotter controlado por el microcontrolador ATmega328P fue evaluada exitosamente en montaje físico. Al ejecutar la programación, el sistema mostró un comportamiento consistente con las especificaciones, moviendo el plotter con precisión para dibujar las figuras predefinidas (triángulo, círculo, cruz, etc.). Los motores respondieron correctamente a los comandos de dirección, realizando los movimientos de los ejes X e Y de manera fluida y precisa.

Durante las pruebas, se verificó que los movimientos del plotter seguían las secuencias programadas para cada figura.

El montaje físico confirmó estos resultados, permitiendo la correcta visualización de las figuras generadas. En la Fig. 21, se presentan las figuras generadas por el plotter, evidenciando la precisión de los movimientos y la correcta ejecución del programa.

En la práctica, el comportamiento observado en el plotter fue fluido y sin fallos, dibujando con exactitud las formas solicitadas. Los tiempos de ejecución y la precisión de los movimientos se ajustaron adecuadamente.

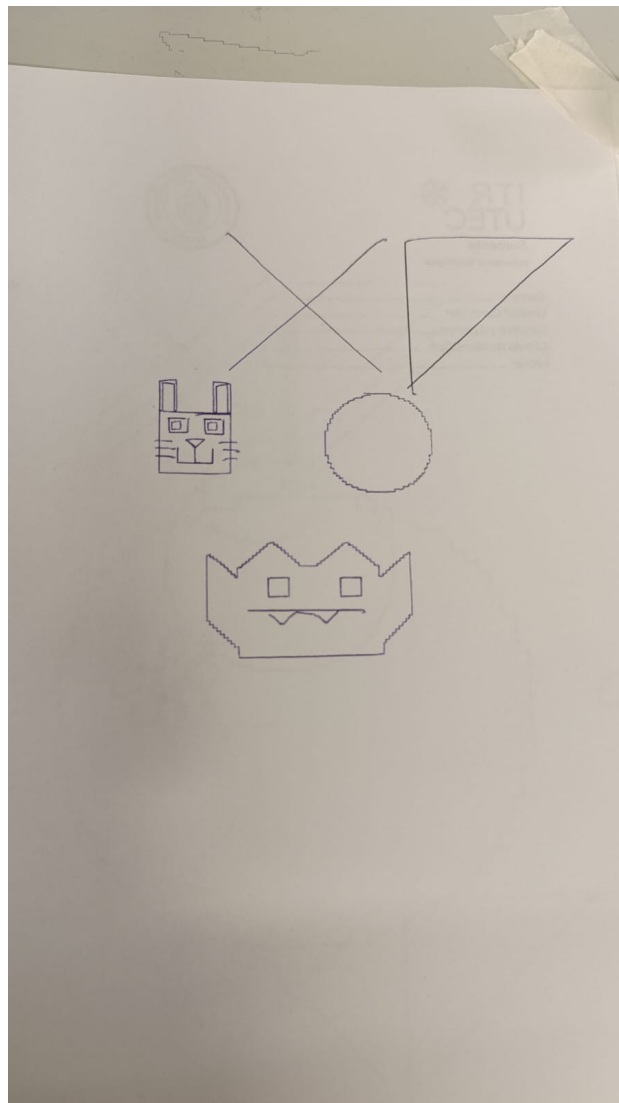


Figura 21: Figuras generadas por el plotter.

VI-B. Sistema de Control de Temperatura

La implementación del sistema de control de temperatura fue evaluada con éxito tanto en simulación como en el montaje físico. En la simulación realizada en Proteus, el comportamiento del sistema fue adecuado, validando que el microcontrolador ATmega328P generaba las señales PWM correspondientes para controlar el ventilador en función de la temperatura medida por el LM35. Durante las pruebas, se observó que el ciclo útil del PWM se ajustaba correctamente conforme a las variaciones de la temperatura, activando el ventilador de manera proporcional.

En el montaje físico, se verificó que el TIP31, como interruptor de potencia para el calefactor, actuaba correctamente al encenderse cuando la temperatura descendía por debajo del umbral inferior, y apagándose cuando se alcanzaba el umbral superior. El ventilador controlado por PWM ajustaba su velocidad de forma precisa en función de la temperatura medida, demostrando una respuesta fluida al cambio de la

señal de entrada.

El sistema fue validado utilizando una interfaz en *Python*, que recibió los datos de temperatura y los mostró en tiempo real en una gráfica. En la Fig. 22, se muestra el montaje físico del sistema en protoboard, mientras que en la Fig. 23 se presenta la gráficas generadas en *Python*, una muestra cómo varía la temperatura en función del tiempo, como también otras 2 que indican el PWM tanto del Heater como del ventilador.

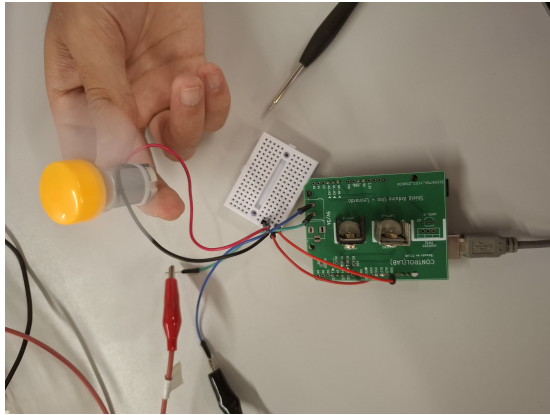


Figura 22: Montaje físico del sistema de control de temperatura

que el motor respondiera correctamente a los cambios en el valor del potenciómetro, logrando un control de velocidad proporcional. Se observaron movimientos relativamente suaves y casos aislados de saltos, lo que indicaba que la señal PWM estaba siendo aplicada de manera eficiente.

El sistema fue validado utilizando una interfaz en *Python*, que permitió visualizar en tiempo real el valor del potenciómetro y el ciclo útil del PWM, graficando la relación entre ambos. En la Fig. 24, se muestra el montaje físico del sistema con el motor y el L298N, mientras que en la Fig. 25 se presenta la gráfica generada en *Python*, donde se observa cómo varía el valor del potenciómetro de referencia, el potenciómetro medido y la activación del PWM.

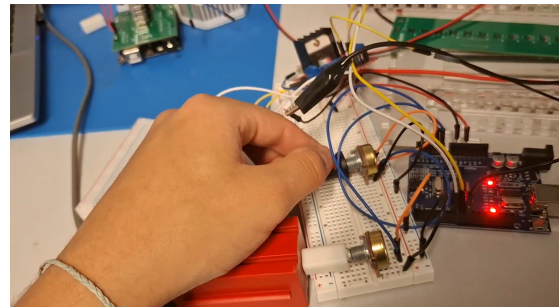


Figura 24: Montaje físico del sistema de control de motor

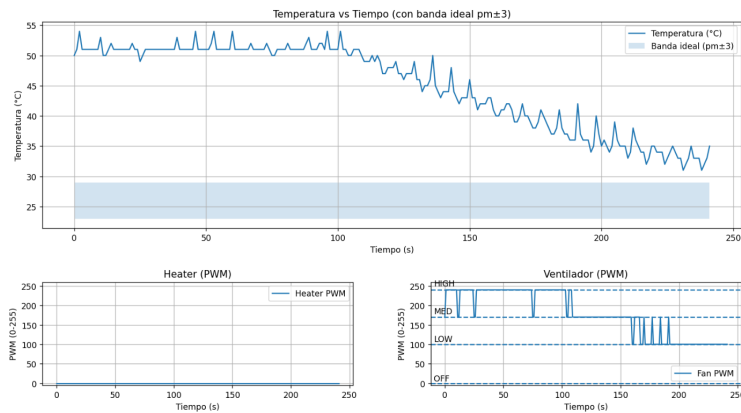


Figura 23: Gráficas generadas en Python en tiempo real

VI-C. Control de Motor

La implementación del sistema de control de motor con referencia analógica fue evaluada con éxito tanto en simulación como en montaje físico. En la simulación realizada en *Proteus*, el comportamiento del sistema fue adecuado, y el motor controlado por PWM respondió correctamente a los cambios en el valor del potenciómetro. El ciclo útil de la señal PWM se ajustaba de forma precisa en función de las variaciones del valor analógico proporcionado por el potenciómetro, permitiendo un control suave de la velocidad del motor.

En el montaje físico, el motor fue controlado mediante el puente H L298N, el cual recibió la señal PWM generada por el ATmega328P para ajustar la velocidad del motor. Se verificó

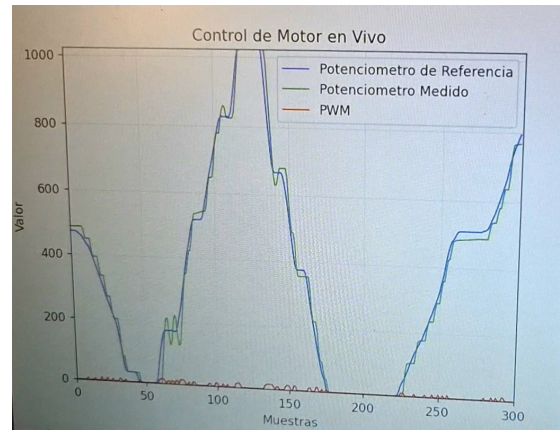


Figura 25: Gráfica generada en Python en tiempo real

VI-D. Matriz RGB con Joystick

La implementación del control de la matriz RGB con joystick fue exitosa tanto en el montaje físico como en la ejecución del código. El sistema respondió adecuadamente a los movimientos del joystick, moviendo el LED dentro de la matriz de 8x8 en las direcciones correspondientes a los ejes X e Y. Cada vez que se movía el joystick, el LED se desplazaba a la nueva posición, mostrando una transición suave y precisa entre las ubicaciones.

El cambio de color se ejecutó correctamente al presionar el botón del joystick, lo que activó el generador de colores aleatorios. Se observó que el LED cambiaba de color de

manera correcta, con colores generados aleatoriamente para cada pulsación.

A pesar de que el sistema funcionó correctamente, se observó que en las zonas cercanas al centro de la matriz, donde el joystick se encuentra en su posición neutral, el movimiento del LED era menos sensible. Esto se debe a la configuración de la zona muerta, que evitó que el LED se moviera accidentalmente cuando el joystick estaba en su punto de reposo. Esta configuración fue crucial para mejorar la precisión del sistema y evitar movimientos erráticos del LED.

Durante las pruebas, también se verificó que el sistema manejara correctamente los límites de la matriz, evitando que el LED se moviera fuera de la pantalla. Cuando el LED alcanzó un límite (ya sea en el eje X o Y), el sistema lo bloqueó y evitó que continuara moviéndose fuera de la matriz, asegurando que permaneciera dentro del rango de visualización de 8x8.

En la Fig. 26 se muestra una captura del montaje de la matriz de LEDs.

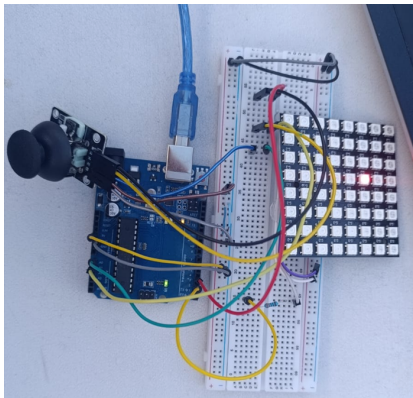


Figura 26: Resultado final del control de matriz RGB con joystick.

VI-E. Cerradura RFID

La implementación del sistema de cerradura RFID fue evaluada con éxito tanto en simulación como en montaje físico. El sistema funcionó correctamente, permitiendo la autenticación de usuarios mediante la lectura de tarjetas RFID y la gestión de acceso a través de la pantalla LCD y los LEDs indicadores.

En la simulación, se verificó que el módulo RFID RC522 leyera adecuadamente las tarjetas y enviara los datos al microcontrolador ATmega328P a través del protocolo SPI. La validación de los datos se realizó comparando el UID de las tarjetas con los valores almacenados en la memoria EEPROM del microcontrolador. Los LEDs indicadores funcionaron correctamente, activando el LED verde cuando la tarjeta era válida y el LED rojo cuando la tarjeta no era reconocida.

En el montaje físico, se observó que la lectura de las tarjetas RFID se realizaba sin problemas, con un rango de lectura adecuado para las tarjetas. La pantalla LCD mostró correctamente los mensajes de "Acceso permitido" y "Acceso denegado", y el sistema respondió de manera rápida y precisa

a las entradas del usuario. Además, el sistema almacenó correctamente las credenciales en la memoria EEPROM, lo que permitió mantener la base de datos de tarjetas incluso cuando el dispositivo fue apagado.

En la Fig. 27, se muestra el montaje físico del sistema en protoboard, con el lector RFID, la pantalla LCD y los LEDs indicadores conectados. En la Fig. 28, se presenta el funcionamiento del sistema, con el acceso permitido y el mensaje en la pantalla LCD.

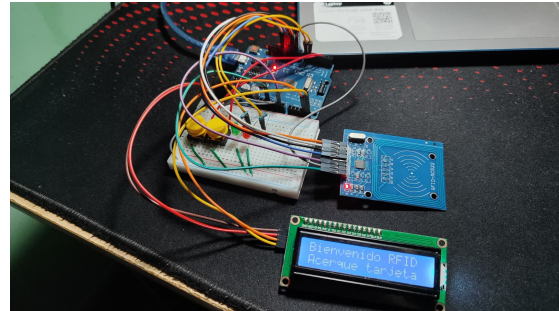


Figura 27: Montaje físico del sistema de cerradura RFID con lector, LCD y LEDs indicadores.

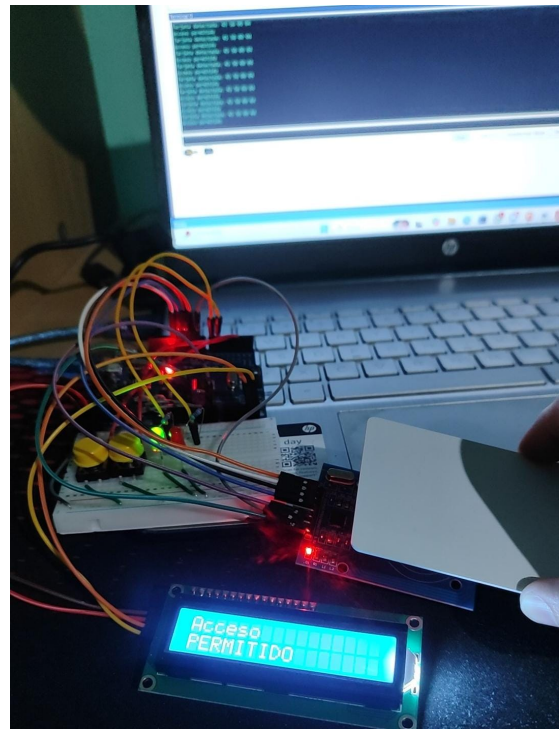


Figura 28: Funcionamiento del sistema de cerradura RFID: acceso permitido y mensaje en la pantalla LCD.

El comportamiento observado en la práctica coincidió con el esperado según los resultados de la simulación. El sistema detectó correctamente las tarjetas autorizadas y activó el mecanismo de acceso sin errores. La comunicación entre el lector RFID, el microcontrolador y los periféricos fue fluida, y la respuesta a las entradas fue inmediata.

El sistema también gestionó adecuadamente los intentos fallidos, bloqueando el acceso temporalmente después de tres intentos incorrectos, tal como se había planeado. Este comportamiento fue verificado con el uso del buzzer y el LED rojo, que indicaron el estado de alerta tras varios intentos incorrectos.

VII. CONCLUSIÓN

En este laboratorio, se implementaron con éxito diversos sistemas de control y adquisición de datos utilizando el microcontrolador ATmega328P, alcanzando los objetivos establecidos al inicio. Cada uno de los sistemas desarrollados, como el plotter controlado por el microcontrolador, el sistema de control de temperatura, el control de motor DC y el sistema de matriz RGB con joystick, cumplió con su propósito, demostrando la efectividad de la integración de sensores, actuadores y comunicación serial en proyectos de control digital.

El plotter respondió correctamente a los comandos, trazando figuras con precisión, lo que validó tanto la programación de los motores como la interacción con el sistema de control. El sistema de control de temperatura mostró un funcionamiento adecuado, con una respuesta estable del ventilador y calefactor al control por PWM, asegurando un rango de temperatura constante. Por su parte, el control de motor DC permitió un manejo eficiente de la velocidad mediante el ajuste de la señal PWM, y el sistema de matriz RGB con joystick facilitó un control preciso del movimiento y el cambio de color del LED en la matriz.

Las pruebas de validación demostraron que todos los sistemas operaban según lo esperado, con los mecanismos de control respondiendo de manera confiable a los parámetros de entrada. A través de este laboratorio, se afianzaron los conocimientos sobre programación en C, comunicación serial, control de motores y la integración de diferentes periféricos para la creación de sistemas embebidos funcionales.

En general, este laboratorio permitió consolidar los principios fundamentales de control digital, procesamiento de señales y comunicación entre dispositivos, siendo una experiencia educativa integral en el ámbito de las tecnologías de microprocesamiento.

VIII. BIBLIOGRAFÍA

REFERENCIAS

- [1] Atmel. *ATmega328P Datasheet*. Disponible en: <https://www.microchip.com/wwwproducts/en/ATmega328P>
- [2] National Semiconductor. *LM35 Precision Centigrade Temperature Sensors Datasheet*. Disponible en: <https://www.ti.com/lit/ds/symlink/lm35.pdf>
- [3] STMicroelectronics. *L298N Dual H-Bridge Motor Driver Datasheet*. Disponible en: <https://www.st.com/resource/en/datasheet/l298.pdf>
- [4] WorldSemi. *WS2812B Intelligent Control LED Driver Datasheet*. Disponible en: <https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf>
- [5] NXP Semiconductors. *RC522 RFID Reader Datasheet*. Disponible en: <https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf>
- [6] Labcenter Electronics. *Proteus Design Suite*. Disponible en: <https://www.labcenter.com>
- [7] Python Software Foundation. *Python Programming Language*. Disponible en: <https://www.python.org/>
- [8] APMonitor. *Arduino Temperature Control*. Disponible en: <https://apmonitor.com/pdc/index.php/Main/ArduinoTemperatureControl>
- [9] Tocci, R. J., Widmer, N. S., & Moss, G. L. (2011). *Sistemas digitales: principios y aplicaciones* (11ª ed.). México: Pearson Educación.
- [10] Mano, M. M., & Ciletti, M. D. (2017). *Digital Design* (6th ed.). Boston: Pearson.
- [11] Floyd, T. L. (2013). *Digital Fundamentals* (11th ed.). Boston: Pearson.

ENLACE AL REPOSITORIO

<https://github.com/Hiato27/Tec.Micro>